

I - Annexe – Enoncés de TP (angular avancé)

1. Préparations et installations

Eventuelle installation de nodeJs et npm (si nécessaire):

Si **node -v** et **npm -v** se sont pas des commandes reconnues (dans un terminal CMD ou autre) alors télécharger et installer **nodeJs** (pour windows 64 bits ou autre en version LTS). Relancer ensuite un nouveau terminal et lancer **npm -v** pour vérifier.

Eventuelle installation de typescript (si nécessaire):

Si **tsc -v** est une commande inconnue (dans un terminal), alors lancer la commande suivante :

```
npm install -g typescript
```

Installation de angular-cli :

Si **ng -help** est une commande non reconnue , alors lancer la commande suivante :

```
npm install -g @angular/cli
```

Installation du cli nx :

Si **nx --version** est une commande non reconnue , alors lancer la commande suivante :

```
npm add --global nx
```

Installer si nécessaire l'IDE **Visual Studio code**.

2. TD "création d'un workspace Nx"

Créer un workspace nx en mode "angular-monorepo"

Voici quelques choix possibles :

- ✓ Application name · **my-nx-workspace**
- ✓ Which bundler would you like to use? · **esbuild**
- ✓ Default stylesheet format · **css**
- ✓ Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? · **Yes**
- ✓ Which unit test runner would you like to use? · **vitest-angular**
- ✓ Test runner to use for end to end (E2E) tests · **playwright ou cypress**
- ✓ Speed up GitHub Actions, GitLab CI, and more with Nx Cloud? · **skip**
- ✓ Help improve Nx by sharing your usage data? · **No**

NB :

- *Sans être compliquée, cette phase de préparation est assez longue ...*
- *Lorsque le workspace nx aura été créé , on pourra le charger/ouvrir (via "openFolder") dans VisualStudioCode ou bien un IDE équivalent.*

3. TP "début application principale"

Au sein du workspace nx créé précédemment , créer un nouveau projet de type "**angular:app**" (ex : **my-app**) via une commande adéquate de nx.

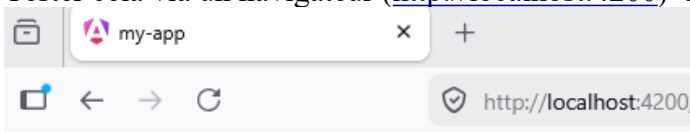
Au départ , esbuild et pas encore besoin de SSR .

Attention : Si la commande "nx show projects" n'arrive pas à trouver/afficher le projet my-app il faudra peut être supprimer ou renommer le fichier .gitignore du répertoire my-nx-workspace .

Structurer le début de cette application avec :

- un sous composant "header" (qui comportera quelques liens de navigation : vers welcome et autres), via le .css on appliquera une couleur de fond "lightcyan" ou autre sur header .
- un composant "welcome" que l'on atteindra via la route par défaut

Tester cela via un navigateur (<http://localhost:4200>) après avoir lancer **nx serve my-app**



header - [welcome](#)

welcome works!

4. TP "Service HTTP avec RxJs"

Phase1 :

- Coder un service angular "ZipService" qui permettra d'invoquer l'api rest <http://api.zippopotam.us/fr/75001> ou autre
- Injecter ce service dans un nouveau composant angular (ex : ZipCurrencyComponent) qui affichera au minimum les données d'un lieu selon la valeur de son code postal
- Ajouter de quoi naviguer vers le "ZipCurrencyComponent" et tester le bon fonctionnement

header - [welcome](#) [zip](#)

zippopotam api call

zip :
data : { "country": "France", "country abbreviation": "FR", "post code": "75001",
"places": [{ "place name": "Paris 01 Louvre", "longitude": "2.3417", "latitude":
"48.8592", "state": "Île-de-France", "state abbreviation": "A8" }] }

éléments techniques à mettre en place :

- classes ou interfaces ZippopotamResponse et ZippopotamPlace dans common/data/Zippopotam.ts selon structure de la réponse à <http://api.zippopotam.us/fr/75001>
- provideHttpClient() dans app.config.ts
- inject(HttpClient) et méthode getZippopotamData\$() dans ZipService
- importation de FormsModule et JsonPipe dans ZipCurrencyComponent

- inject(ChangeDetectorRef); et this.changeDetector.markForCheck(); dans ZipCurrencyComponent

Phase2 (facultative mais conseillée):

- Ajouter de nouvelles fonctionnalités dans "ZipCurrencyComponent" et des services en arrière plan qui exploiteront certains opérateurs RxJs (filtrage ou tri ou autres combinaisons)

Quelques Idées/Suggestions :

currencies

**** base : EUR / Euro ****

USD : US Dollar , JPY : Japan Yen ,

GBP : Great Britain Pound (Livre sterling) ,

DKK : Danemark Couronne/Krone ,

CNY: ChiNa Yuan, KRW : KoRea Won , ...

order:

filteringCurrencyCodes:

```
currencies : [ { "code": "CNY", "value": 7.902704 }, { "code": "DKK", "value": 7.472058 }, {  
"code": "EUR", "value": 1 }, { "code": "GBP", "value": 0.862938 }, { "code": "JPY", "value":  
184.929496 }, { "code": "KRW", "value": 1751.743686 }, { "code": "USD", "value": 1.16306 }  
]
```

Le web service public fixer.io retourne les taux de change des principales monnaies.
Son url est :

let apiKey = "26ca93ee7fc19cbe0a423aaa27cab235"; //ici avec apiKey de didier

let wsUrl = `http://data.fixer.io/api/latest?access\_key=\${apiKey}`

La réponse de ce web service est dans ce format :

```
{ "success": true, "timestamp": 1779774247, "base": "EUR", "date": "2026-05-26", "rates":  
{ "AED": 4.271343, ..., "EUR": 1, ..., "GBP": 0.862938, ..., "USD": 1.16306, ..., "ZWL": 374.504938 } }
```

Via une combinaison adéquate d'opérateurs RxJs (dans partie .pipe()) , on pourra (par exemple) :

- afficher la réponse (brute, d'origine) dans la console du navigateur
- transformer les taux de change du format d'origine vers un format liste de "Currency"
- trier soit par code de monnaie croissant , soit par valeur du taux de change
- filtrer les codes de monnaies intéressantes (ex : EUR,USD,GBP)

```
{ "EUR": 1, ..., "GBP": 0.862938, ..., " } { "code": "EUR", "value": 1 }, { "code": "GBP", "value":
```

USD":1.16306 }	0.862938 }, { "code": "USD", "value": 1.16306 }
export interface CurrencyMap{ [index: string] : number; }	export class Currency { constructor(public code : string, public value : number){} }

Pour tester (facultativement) , un observable d'ordre 2 , on pourra éventuellement enchaîner/combiner via mergeMap() ou autre deux appels de Web Services (dans la partie Zip):

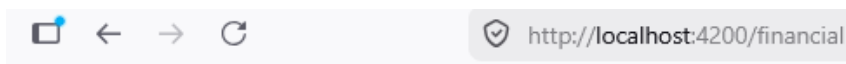
- 1) via un appel au WS "zippopotam" on récupère les coordonnées GPS (longitude , latitude) à partir du codePostal (zip)
- 2) via un appel au WS "openweathermap" on récupère des informations météorologiques à partir des coordonnées géographiques

Exemple d'appel au WS "openweathermap":

```
let apiKey = "27a68278aebec75af9d4fc23d7a68f75"; //ou autre après inscription
//let lat = 48.856614; //latitude (ici de Paris)
//let lon = 2.3522219; //longitude (ici de Paris)
let wsUrl = `https://api.openweathermap.org/data/2.5/weather?lat=${geoCoord.lat}&lon=${geoCoord.lon}&appid=${apiKey}`
return this.http.get<any>(wsUrl);
```

5. TP "composant avec signaux"

- Programmer un nouveau composant "FinancialComponent" qui servira à saisir certains paramètres tels que "nbMonth" , "amount" , "annualRate" et qui calculera et affichera la somme à rembourser chaque mois pour rembourser un emprunt à mensualité constante .
- On placera la formule de calcul dans un service angular "FinancialService" à injecter.
- On utilisera au maximum des signaux angular (ex : signal() , computed() , ...)



header - [welcome](#) [zip](#) [financial](#)

financial computation

nbMonth:
amount :
annualRate (%):

monthlyPayment: 942.6990170396044

Formule de calcul :

```
monthlyPaymentValue(nbMonth : number, amount : number , annualRate : number):number {
  let monthRate = (annualRate / 12)/100;
  return (amount * monthRate) / ( 1 - Math.pow(1+monthRate, -nbMonth));
}
```

6. TP avec Observable et signal

Au sein du composant "**ZipCurrencyComponent**", modifier au moins une partie du code de façon à ce qu'une valeur observable issue d'un appel HTTP soit transformée en signal prêt à être affiché.

7. TP "composants réutilisables" dans projet my-lib

- Au sein du workspace Nx, créer un nouveau sous projet "my-lib" de type angular:lib
- Au sein de ce projet "my-lib" créer au moins un composant réutilisable
- Au sein du projet "application principale", ajouter quelque part une utilisation d'un des composants réutilisables de "my-lib"

Idee de composants réutilisables :

- selector (un choix de type string en output() à partir d'une liste d'éléments en input())
- togglePanel avec titre et sous composant paramétrable et panelOpenState de type model()
- confirmDialog : boîte de dialogue de confirmation (message, "yes", "no", ...)

configuration et éléments à mettre en place :

- src/index.ts de my-lib
- fichier package.json dans my-lib et my-app (avec partie "dependencies")
- imports nécessaires (ex : import {XyzComponent} from '@my-nx-workspace/my-lib';)

Exemple de résultats :

financial computation

☐ loan computing

nbMonth:
 amount :
 annualRate (%):

 monthlyPayment: 942.6990170396044

☒ loan computing

<lib-selector> appliqué dans Welcome:	couleurChoisie = green reset to blue (with confirmDialog)
---------------------------------------	---

welcome to my-app simple-selector choix couleur • red • green • blue couleurChoisie = green	reset couleurChoisie to blue ? No/Cancel <input checked="" type="button" value="Yes/Ok/Confirm"/>
---	--

8. TP "lazy loading de composant et sous-routes"

1. Mettre en place un "lazy loading de composant" sur "ZipCurrencyComponent"
2. On testera/visualisera si possible le comportement "lazy"
3. Décomposer "ZipCurrencyComponent" en "partie menu avec sous routes" + nouveaux sous composants tels que "ZipComponent" + "CurrencyComponent"
4. Réadapter le LazyLoading via un "loadChildren" basé sur m.zipCurrencyRoutes

header - [welcome](#) [zip_currency](#) [financial](#) [person](#) [catFact](#)

zippopotam , weather and currency call

[zip](#) [currency](#) ...

zip , geoCoord and weather

zip :

data :
geoCoord :
wheater :

9. TP facultatif "architecture micro-frontend"

- Au sein du workspace nx , créer un nouveau sous projet "mfe1" de type application angular
- Ajouter les fonctionnalités "federative micro-frontend" au niveau de l'application principale (rôle "host") et du projet "mfe1" (rôle "remote")
- **En cas de soucis de versions → npm install --legacy-peer-deps et si c'est nécessaire certains redémarrages peuvent quelquefois être utiles.**
- déplacer toute la partie "ZipCurrency" (service + composants + sous routes) dans le projet "mfe1"
- ajouter toutes les configurations nécessaires aux communications "micro-frontend"

- tester le bon fonctionnement :
nx serve mfe1 --port=4201
nx serve my-app
<http://localhost:4201> et <http://localhost:4200>
- Attention toSignal() n'est pas toujours bien compatible avec l'api "federative micro-frontend" (selon versions). Revenir si besoin à un code sans .toSignal().

10. TP "formulaire avancé ou dynamique"

Phase 1 :

- Au sein l'application principale , créer un composant "PersonComponent" comportant un formulaire de saisie pour une données de type data.Person (avec "firstname" , "lastname" , email et age) . On pourra au choix s'appuyer sur l'api ReactiveForm/FormBuilder ou bien sur la nouvelle api SignalForm

Phase 2 (facultative) :

- Au sein du sous projet my-lib , créer un nouveau composant réutilisable "DynamicForm"
- Retoucher le composant "PersonComponent" au sein de l'application principale de façon à ce qu'une partie soit déléguée à "DynamicForm" .

En cas d'erreur de validation :

signalForm (person) in responsive tailwind css grid

firstname:

jean

lastname:

Bon

email:

jean.bonxyz.com

Please enter a valid email address

age:

4

You must be at least 18 years old

Lorsque tout est bien saisi :

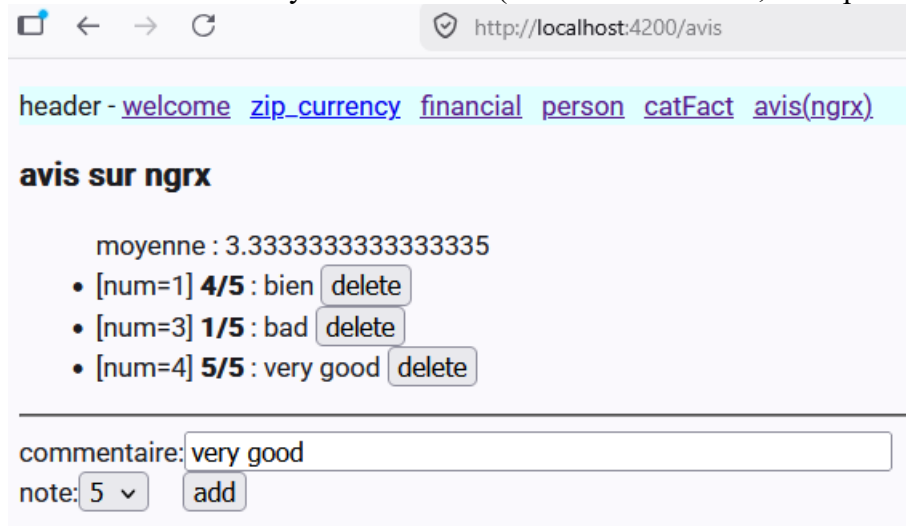
login

message: valeurs

saisies={"firstname":"jean","lastname":"Bon","email":"jean.bon@xyz.com","age":"40"}

11. TP libre "gestion avancée des états (ngrx ou ...)"

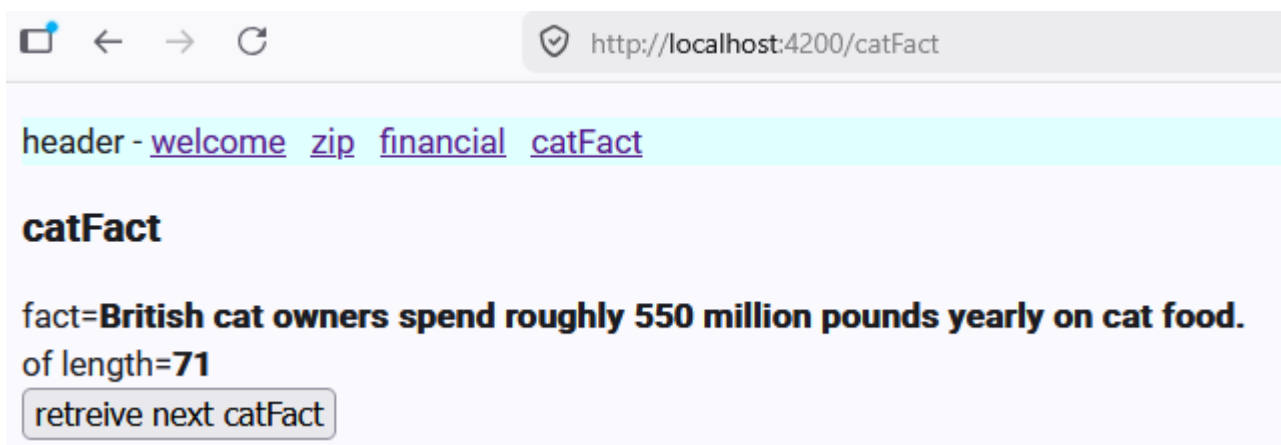
- Ajouter un nouveau sous composant "AvisComponent" (éventuellement/facultativement décomposé en sous composants) qui s'appuiera par exemple sur ***ngrx*** de manière à bien gérer l'état d'une donnée "moyenne des avis" (note allant de 1 à 5 , exemple 3.7)



12. TP "resolver"

La web service public <https://catfact.ninja/fact> renvoie un des faits sur les chats au format suivant : {"fact":"Cats make about 100 different sounds. Dogs make only about 10.", "length":62}

- Programmer hyper rapidement/simplement "CatFactService" + "CatFactComponent" de manière à afficher le résultat du seb service rest <https://catfact.ninja/fact>
- Au sein du service basé sur HTTP+RxJs , ajouter dans la méthode d'appel en mode get un ajout de temporisation (pause) variable (par exemple 2000 ms) via **.pipe()** et l'opérateur **delay(nbMs)**
- Visualiser un effet "navigation vers vide + affichage différé après attente de récupération des données"
- Ajouter un resolver "catFactResolver" au niveau de la route permettant de naviguer vers "CatFactComponent"
- Visualiser navigation retardée + affichage en un seul coup des données pré-chargées .



13. TP "tests unitaires"

- Tester unitairement le service "FinancialService"
Pour un lancement via nx: `nx test my-app --include=**/financial.service.spec.ts`
- Tester unitairement le composant "FinancialComponent" sur la fonctionnalité calcul de mensualité à rembourser.
Pour éviter toute ambiguïté, on ajoutera si besoin des `type="number"` et `name="..."` sur les zones `<input ...>` du composant FinancialComponent et l'affichage du résultat sera encadré dans une balise `<span>` ayant un `id` connu.

14. TP "test e2e"

- Effectuer un test de calcul de mensualité en mode "e2e" soit avec cypress, soit avec playwright

Pour un lancement via nx et playwright :

`npx playwright install` (au préalable si besoin)

`nx e2e my-app-e2e`

et pas absolument besoin de lancer préalablement `nx serve my-app`.

15. TP "angular SSR"

- Ajouter la fonctionnalité "@angular/ssr"
- Ajuster si besoin certains points (ex : accès indirect à sessionStorage ou localStorage)
- effectuer quelques "build" et tests

16. TP libre "selon intérêts (PWA ou OAuth2 ou i18n)"

Ajouter et tester au moins une des fonctionnalités suivantes (à ajouter à l'application principale) :

- ProgressiveWebApp
- internationalisation
- authentification OAuth2 et gardien
- ...